

Chapter 6: Transport layer and protocols

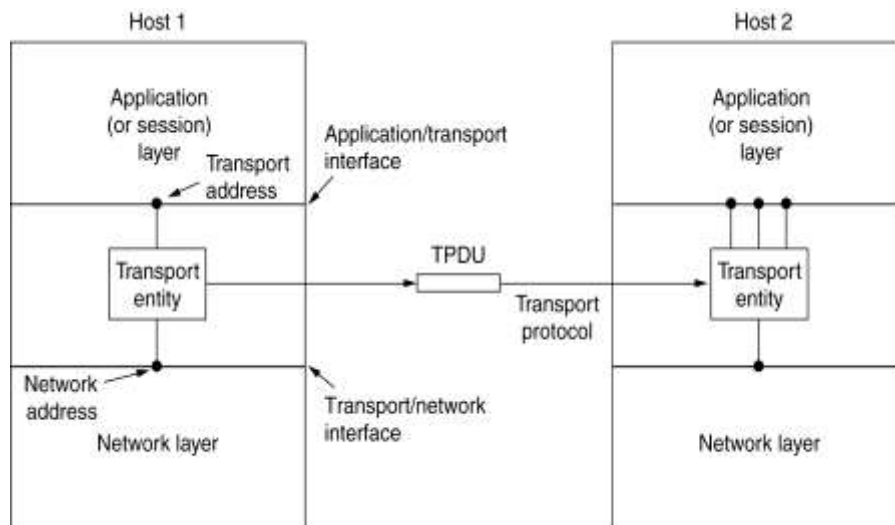
Why do we need a Transport Layer?

With a Network Service Provider you can exchange packets between hosts (e.g. PCs), these hosts are uniquely identified by their network address (e.g. IP address). As a user you may want to send and receive email, browse the web, logon to another host. So, you may want to run several programs or processes. The transport layer allows for processes or applications to communicate with each other. Networks (and the network layer) is under control of a network operator. The network service users have no control over it. So, if something goes wrong, a user can do nothing.

The transport service is what users can add to improve the reliability of the service provided by the network. Functions that you can encounter in the Transport Layer are:

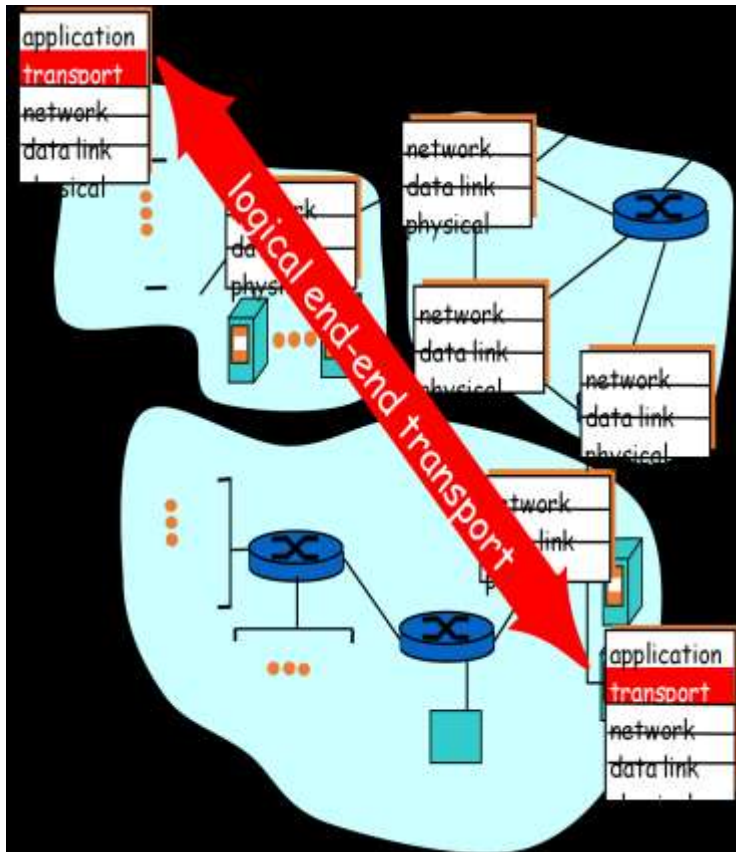
- Error Handling
- Flow Control
- Multiplexing
- Connection Set-up and Release
- Congestion Handling
- Segmentation and Reassembly
- Addressing

Services Provided to the Upper Layers



- The ultimate goal of the transport layer is to provide efficient, reliable, and cost-effective data transmission service to its users, normally processes in the application layer.
- The software and/or hardware within the transport layer that does the work is called the transport entity.

- It provide *logical communication* between application processes running on different hosts
- Transport protocols run in end systems
 - send side: breaks application messages into segments, passes to network layer
 - receive side: reassembles segments into messages, passes to application layer
- more than one transport protocol available to apps
 - Internet: TCP and UDP



Port Addressing

- Ports are conceptual “points of entry” into a host computer.
- Usually a service is associated with a port (e.g. http on port 80).
- Servers “listen on a port” for connection attempts.
- Ports provide one level of internet security.
 - Each *process that wants to communicate with another process identifies itself to the TCP/IP protocol suite by one or more ports.*
- Popular applications have well-known ports.

- Ranges of Port Number
 - Server has a well-known port (e.g., port 80 for HTTP)
 - Between 0 and 1023 (requires root to use)
 - Allocated to server services by IANA
 - Registered Port
 - Between 1024 and 49151
 - Can be registered for services with the IANA
 - Client picks an unused ephemeral (i.e., temporary) port
 - Between 49152 and 65535
 - Used by client programs

Services	Listens on Port
HTTP	80
Telnet	23
SMTP	25
FTP Control	21
FTP Data	20
SSH	22
POP3	110
IMAP	143

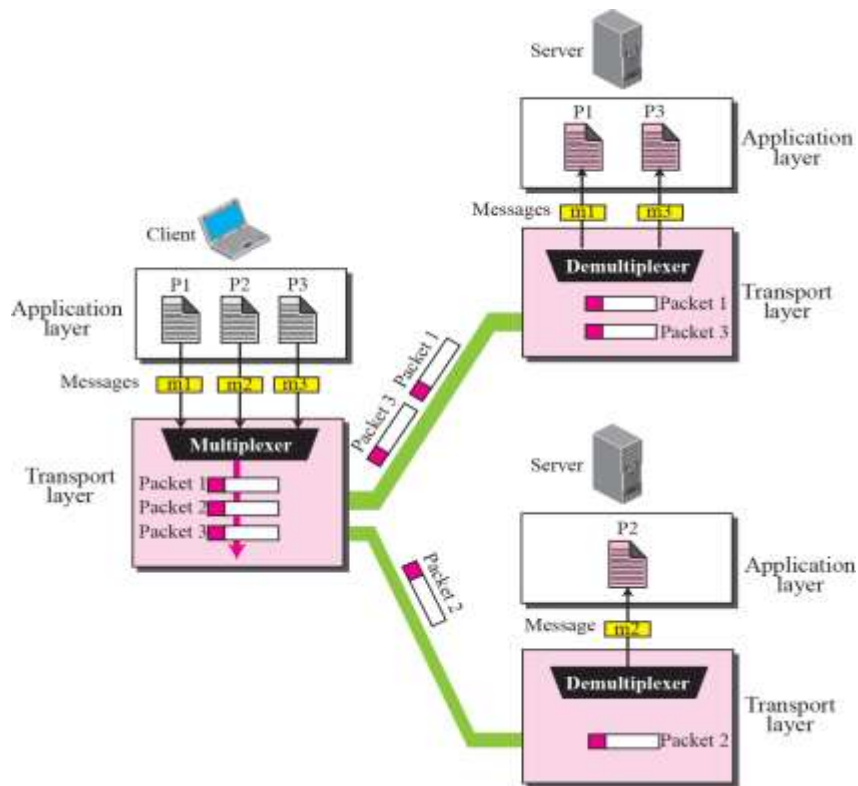
Multiplexing & De-multiplexing

De-multiplexing at rcv host:

It is associated with delivering received segments to correct socket

Multiplexing at send host:

It is concerned with gathering data from multiple sockets, enveloping data with header (later used for de-multiplexing)



Connectionless de-multiplexing

- UDP socket identified by two tuple:
 - (destination IP address, destination port number)
- When host receives UDP segment:
 - checks destination port number in segment
 - directs UDP segment to socket with that port number

Connection-oriented de-multiplexing

- TCP socket identified by 4- tuple:
 - source IP address
 - source port number
 - destination IP address
 - destination port number
- Receiving host uses all four values to direct segment to appropriate socket
- Server host may support many simultaneous TCP sockets:
 - each socket identified by its own 4-tuple

- Web servers have different sockets for each connecting Client
 - non-persistent HTTP will have different socket for each request

Transmission Control Protocol (TCP)

TCP Protocol Functions

- Multiplexing
- Error Handling
- Flow Control
- Congestion Handling
- Connection Set-up and release

TCP Transport Service

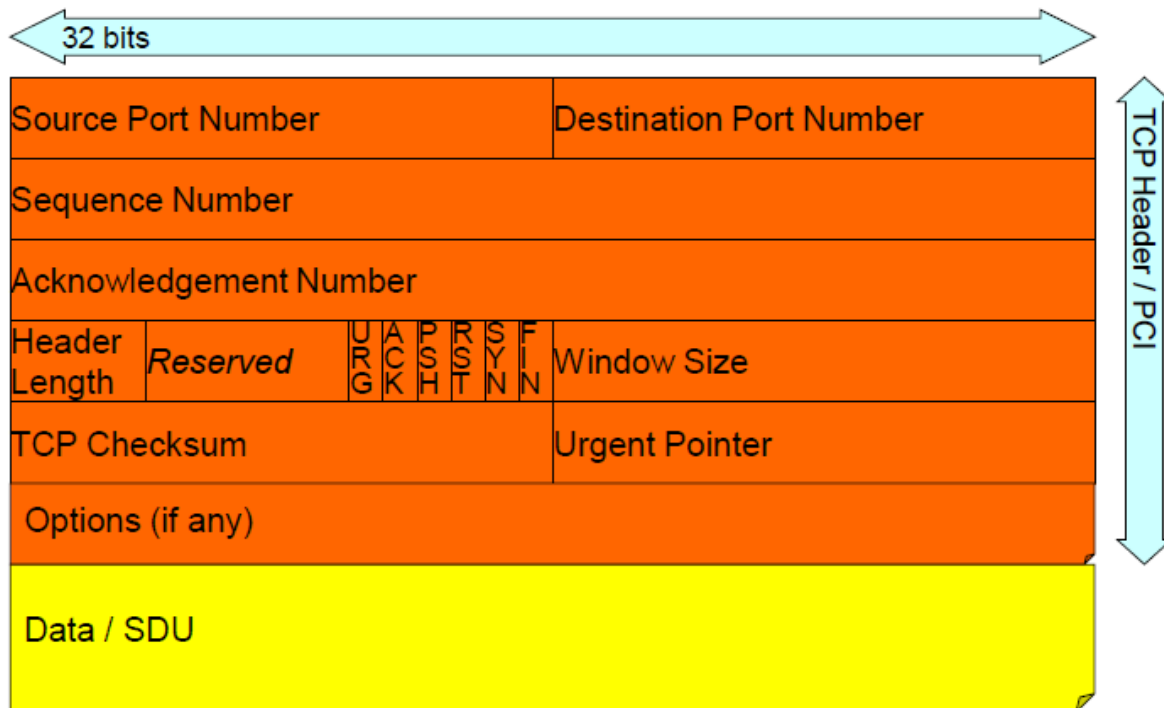
- Connection Oriented (full duplex point-to-point connection between processes).
- Reliable
- In-sequence segment delivery

Well-known TCP ports

Protocol	TCP port
HTTP	80
HTTPS	443
FTP	20,21
SMTP	25
Telnet	23
IMAP	143
POP3	110

TCP Segment

- ✓ Source Port: The 16-bit source port number, used by the receiver to reply.
- ✓ Destination Port : The 16-bit destination port number
- ✓ Sequence Number : The sequence number of the first data byte in this segment.
- ✓ Acknowledgment Number: If the ACK control bit is set, this field contains the value of the next sequence number that the receiver is expecting to receive.
- ✓ Header length: A 4-bit field that represents the header length in multiple of four bytes
- ✓ Reserved: Six bits reserved for future use; must be zero.
- ✓ URG: Indicates that the urgent pointer field is significant in this segment.



- ✓ ACK: Indicates that the acknowledgment field is significant in this segment.
- ✓ PSH: Push function.
- ✓ RST: Resets the connection
- ✓ SYN: Synchronizes the sequence numbers.
- ✓ FIN : No more data from sender.
- ✓ Window size: It specifies the number of data bytes that the receiver is willing to accept.
- ✓ Checksum : A 16-bit field used for error correction.
- ✓ Urgent Pointer : Points to the first data octet following the urgent data.

Opening a connection (3-way handshake):

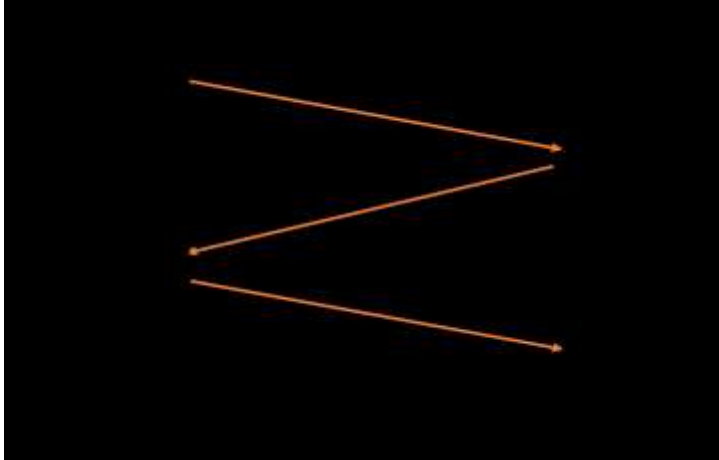
Step 1: client end system sends TCP SYN control segment to server

Step 2: server end system receives SYN, replies with SYN-ACK

- allocates buffers
- ACKs received SYN

Step 3: client receives SYN-ACK

- connection is now set up
- client starts the “real work”



Closing a connection:

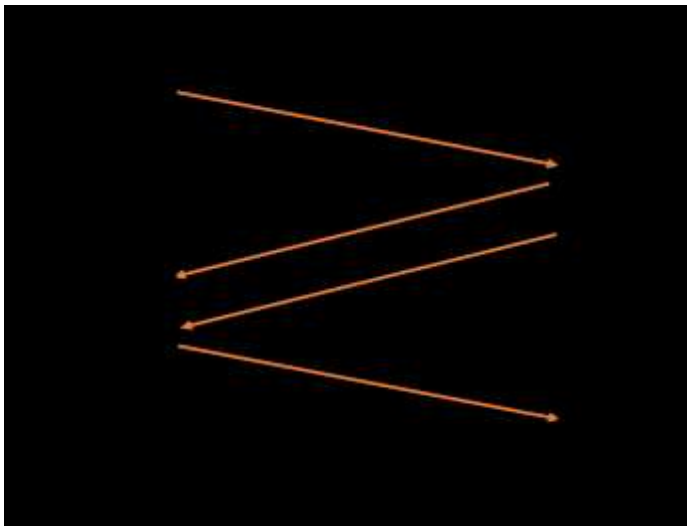
Step 1: client end system sends TCP FIN control segment to server

Step 2: server receives FIN, replies with ACK. Closes connection, sends FIN

Step 3: client receives FIN, replies with ACK.

- Enters “timed wait” - will respond with ACK to received FINs

Step 4: server, receives ACK. Connection closed.



UDP (User Datagram Protocol)

UDP protocol functions are:

- Multiplexing
- Error Detection

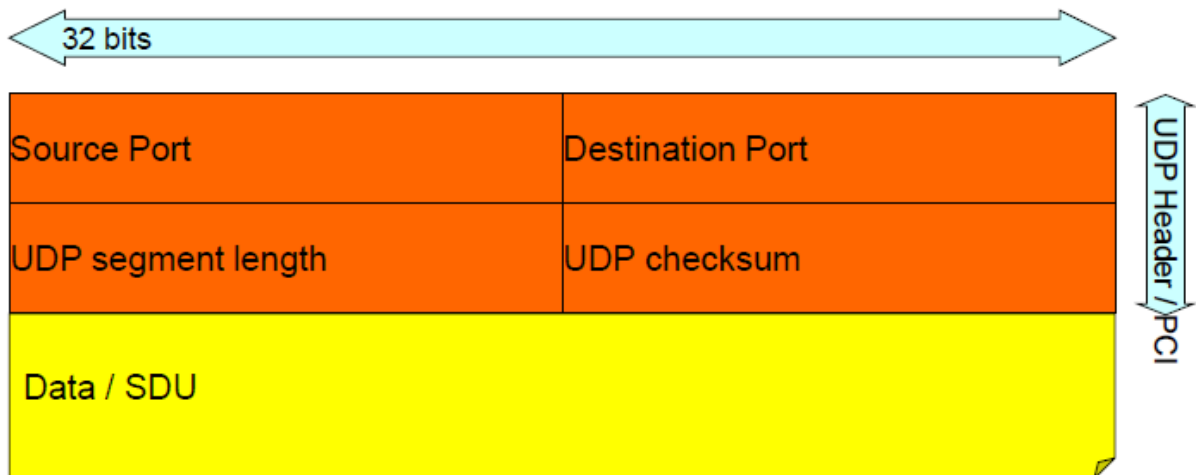
UDP Services:

- Is a connectionless service
- Is unreliable
- Has no in-sequence delivery guarantees

Well-known UDP ports

Protocols	UDP Ports
DNS	53
DHCP	67,68
SNMP	161,162

UDP Segment



Why would anyone use UDP?

- No delay for connection establishment
 - UDP just blasts away without any formal preliminaries
 - avoids introducing any unnecessary delays
- No connection state
 - No allocation of buffers, parameters, sequence numbers, etc.
 - easier to handle many active clients at once
- Small packet header overhead
 - UDP header is only eight-bytes long

Comparison of TCP and UDP

S.N	TCP	UDP
1	Stream Oriented	Datagram Oriented
2	Reliable, Connection-oriented	Unreliable, Connectionless
3	Complex	Simple
4	Only Unicast	Unicast and multicast
5	Uses Windows or Acks	No windows or Acks
6	Full Header	Small header, less overhead
7	Sequencing	No sequencing
8	Used for most internet applications	Useful for only few applications
9	Example: HTTP, FTP, SMTP	Example: DNS, SNMP

Socket

To the kernel, a socket is an endpoint of communication. To an application, a socket is a file descriptor that lets the application read/write from/to the network. All Unix I/O devices, including networks, are modeled as files. Sockets are UNIQUELY identified by Internet address, end-to-end protocol, and port number. Clients and servers communicate with each by reading from and writing to socket descriptors.

Socket programming with TCP

Client must contact server

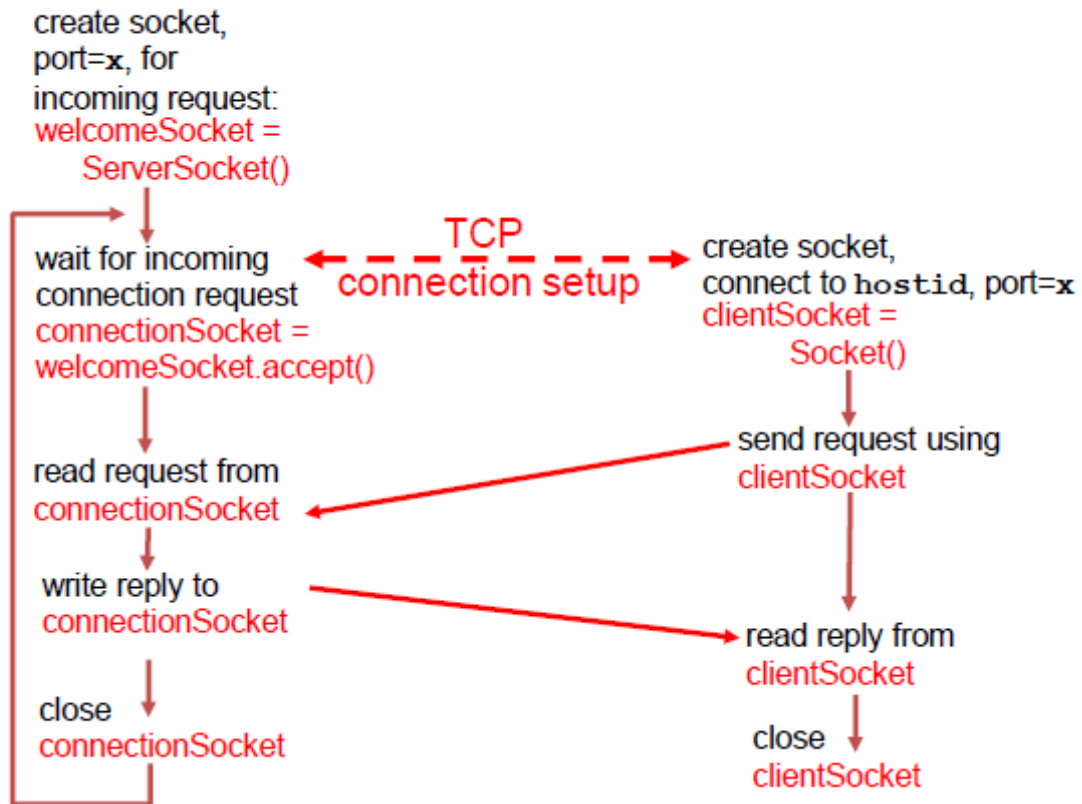
- server process must first be running
- server must have created socket (door) that welcomes client's contact

Client contacts server by:

- creating client-local TCP socket
- specifying IP address, port number of server process
- client establishes TCP connection to server

Server (running on `hostid`)

Client



Socket programming with UDP

No "connection" between client and server

- no handshaking
- sender explicitly attaches IP address and port of destination to each packet
- server must extract IP address, port of sender from received packet to return uppercase sentence to sender

Transmitted data may be received out of order, or lost

Server (running on `hostid`)

Client

